# Machine Learning: Artificial Neural Network

(Slide made by Arindam Mitra)

# Artificial Neural Network

- Neural networks, is a biologically-inspired programming paradigm which enables a computer to learn from observational data.
- For example:
  - The idea is to take a large number of handwritten digits, known as training examples.
  - Then develop a system which can learn from those training examples.
- Furthermore, by increasing the number of training examples, the network can learn more about handwriting, and so improve its accuracy

# Perceptron (Building Blocks)

- Before diving into the artificial neural network let's first see a particular type of a `neuron', known as 'perceptron'.
- A perceptron takes several real inputs, x1,x2,…xn and produces a single binary output.
- The rule to compute the output is very simple:

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{ threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{ threshold} \end{cases}$$

- Where x's and are real numbers.

# Perceptron

- You can use them for real life applications such as 'credit card approval'.

For input $\mathbf{x} = (x_1, \cdots, x_d)$ 'attributes of a customer'

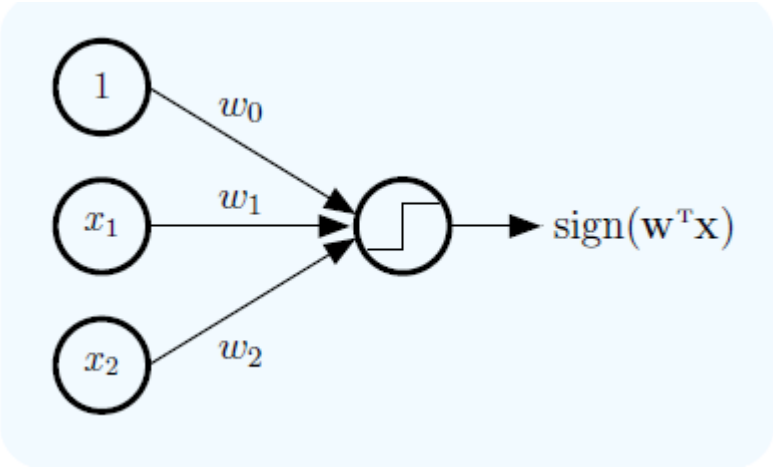
$$\text{Approve credit if} \quad \sum_{i=1}^{d} w_i x_i > \text{threshold},$$

$$\text{Deny credit if} \quad \sum_{i=1}^{d} w_i x_i \leqslant \text{threshold}.$$

# Perceptron

- If your $threshold$ is high that means you are very strict on approving credit cards.
- When should the $w_i$ is positive? (and negative? )
- Thus, we can see by changing the threshold and w's we can produce different functions.

# Diagram

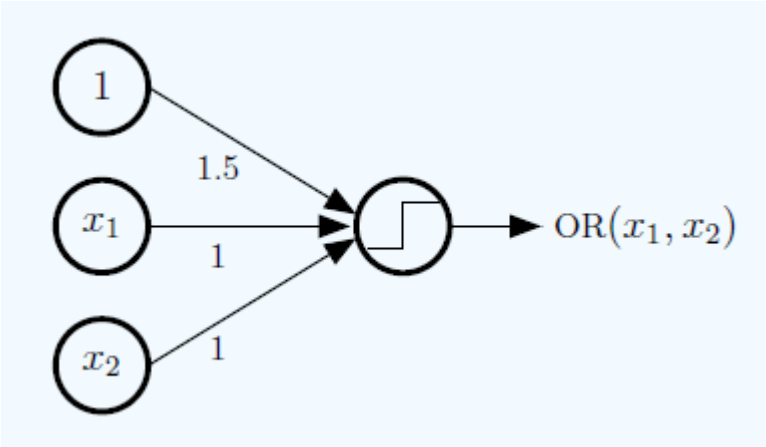



- $w_0 = -threshold$
- Sign(x) = 1 if x>0
  = 0 otherwise

Also called the step function.

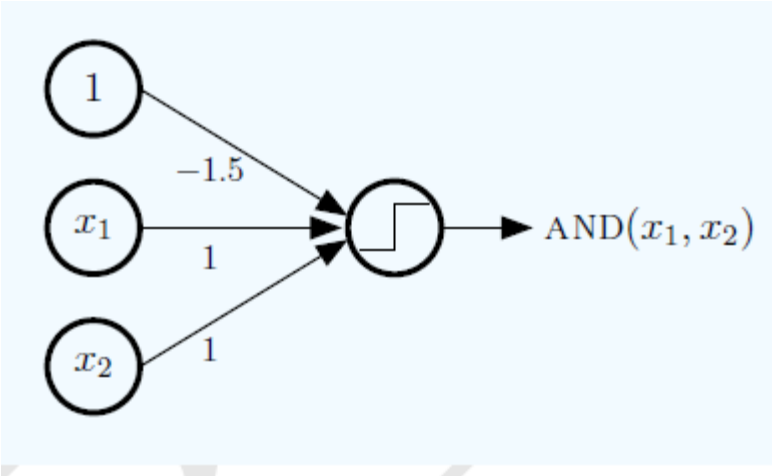
# OR Gates

- Implement a binary OR gate (show w's)
  - Inputs are binary numbers +1/-1 (+1 denotes true and -1 denotes false)
  - Output should be +1/-1
  - Extend it to n-ary OR gate



# AND gates

- Implement a binary AND gate (show w's)
  - Inputs are binary numbers +1/-1
  - Output should be +1/-1
  - Extend it to n-ary AND gate

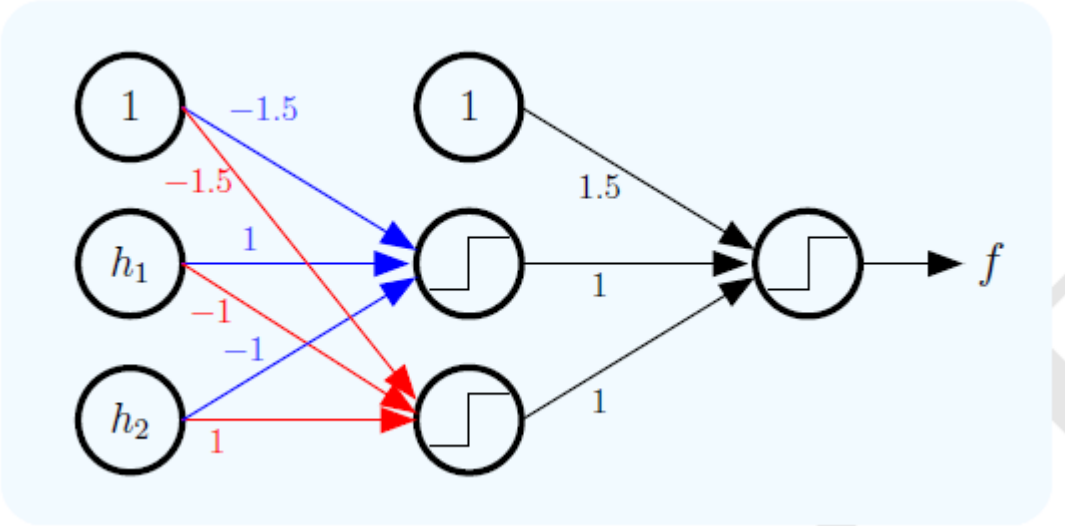


# NAND gates

- What about binary NAND gats?

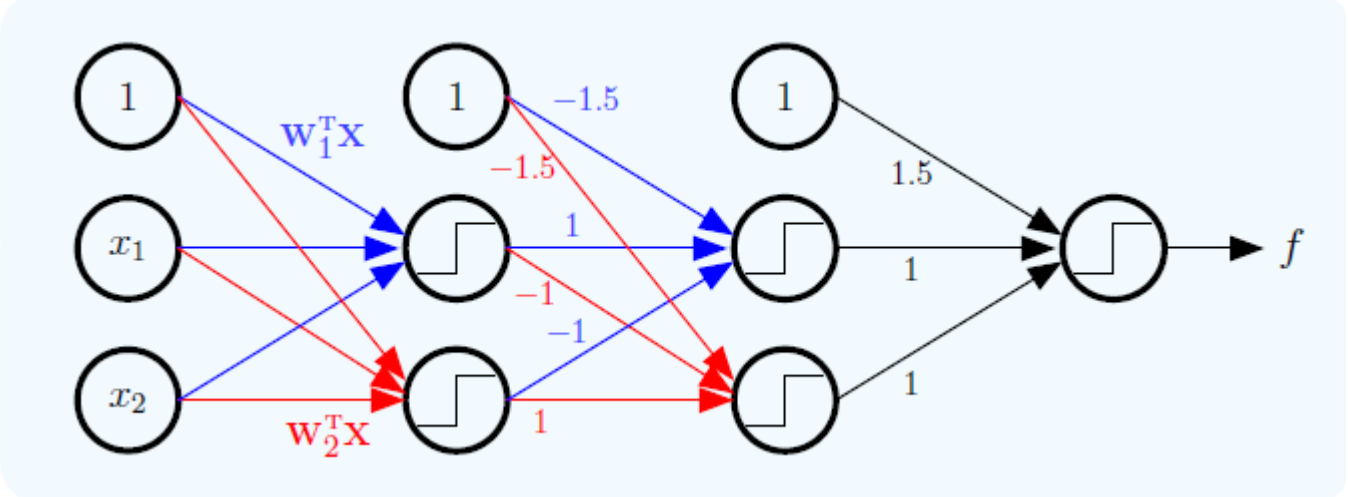
# Can we implement XOR gates

- The answer is No
- However, as we can see if you create a network of such neurons then we can solve the problem.
- Y = h1 xor h2 =  (h1 and not h2) OR (not h1 and h2)



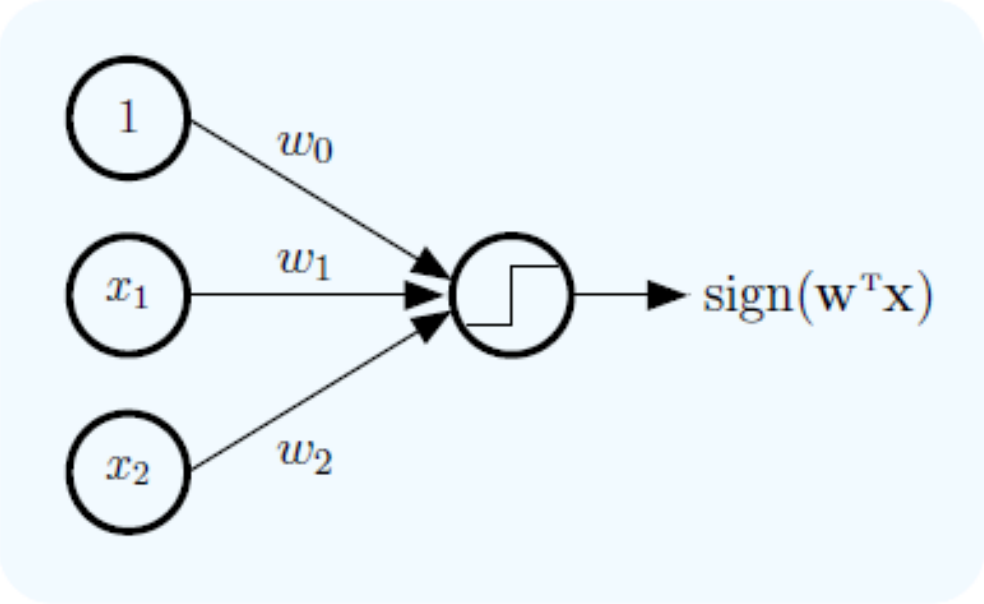
# Exercise

- Show the formula for f



# Multi-Layer Perceptron

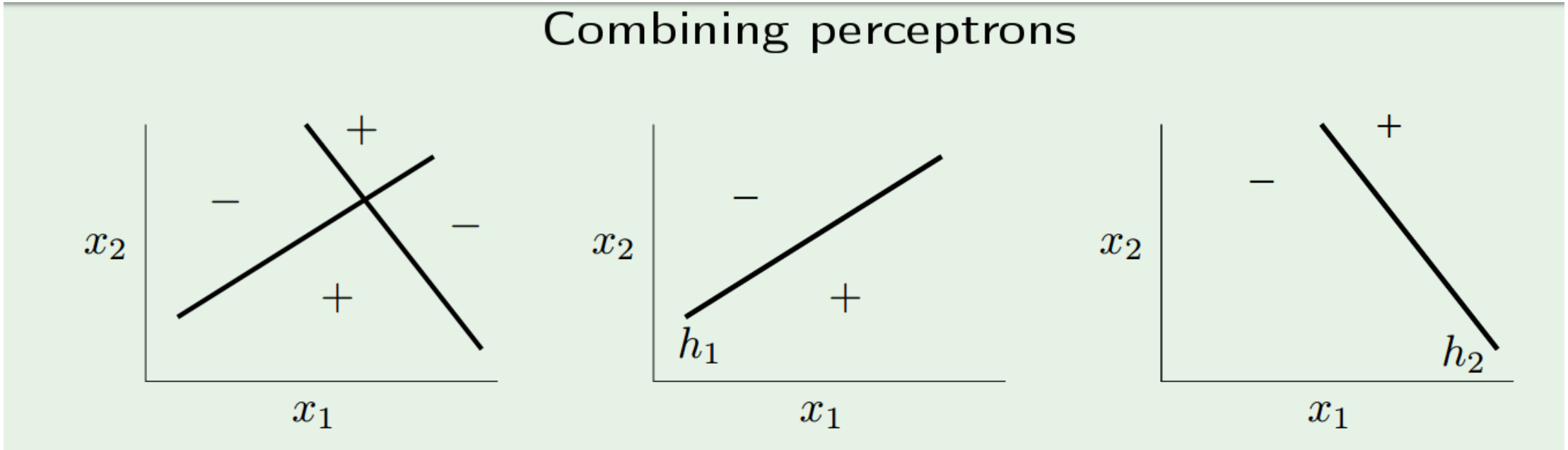
Let's compare the graph form of $f$ with the graph form of the simple perceptron, shown to the right. More layers of nodes are used between the input and output to implement $f$, as compared to the simple perceptron, hence we call it a multi-layer layer perceptron (MLP),. The additional layers are called *hidden layers*.



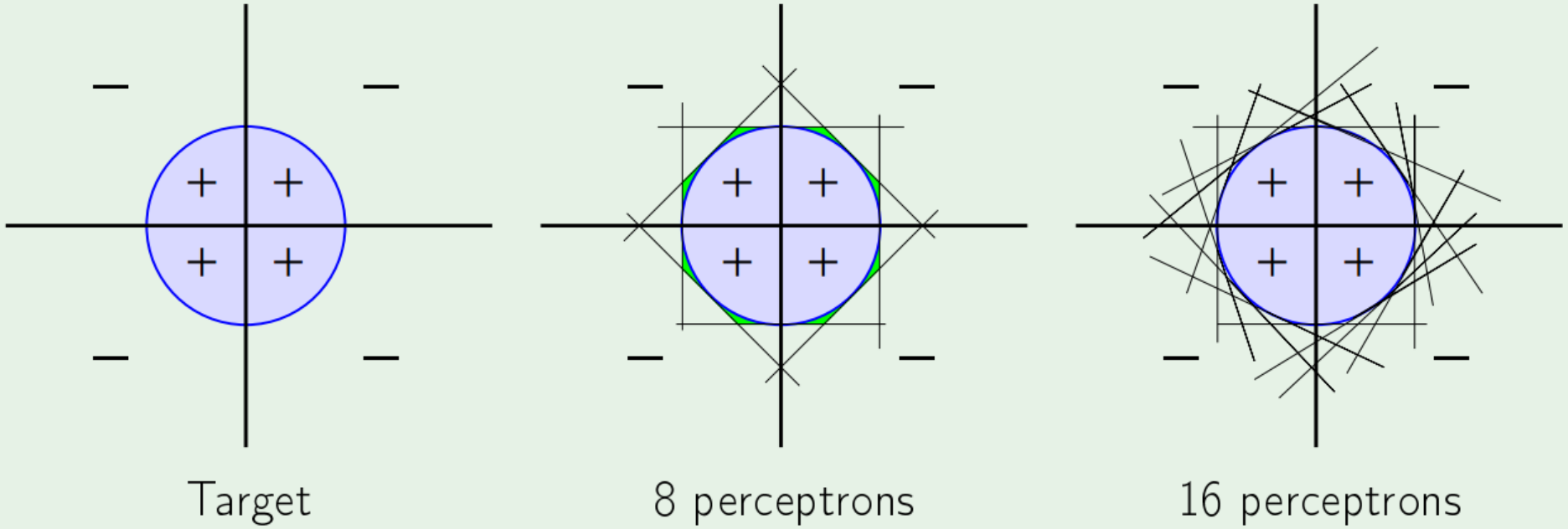
# Multi Layer Perceptrons

Notice that the layers feed forward into the next layer only (there are no backward pointing arrows and no jumps to other layers). The input (leftmost) layer is not counted as a layer, so in this example, there are 3 layers (2 hidden layers with 3 nodes each, and an output layer with 1 node). The simple perceptron has no hidden layers, just an input and output. The addition of hidden layers is what allowed us to implement the more complicated target.

# Graphical Description Of XOR



# A powerful model

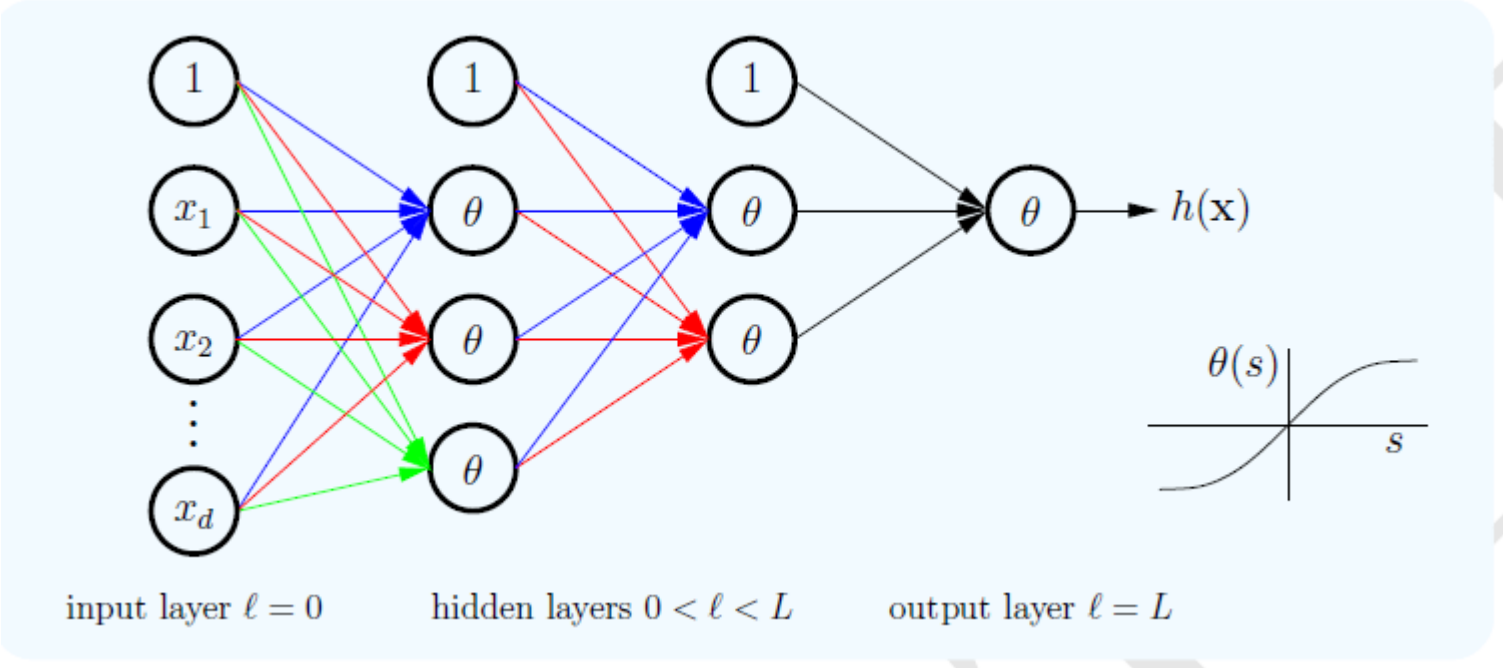


# MLP

- We have thus found a generalization of the simple perceptron that looks much like the simple perceptron itself, except for the addition of more layers.
- We gained the ability to model more complex target functions by adding more nodes (hidden units) in the hidden layers – this corresponds to allowing more perceptrons in the decomposition of f. In fact, a suitably large 3-layer MLP can closely approximate just about any target function, and fit any data set,
- so it is a very powerful learning model.

# Neural Network

- The neural network is our 'softened' MLP.
- Let's begin with a graph representation of a feed-forward neural network.



# Neural Network

- The neural network model M is specified once you determine the architecture of the neural network, that is the dimension of each layer $d = [d(0), d(1), \ldots, d(L)]$ (L is the number of layers).
- A function $f \in M$ is specified by selecting weights for the links.

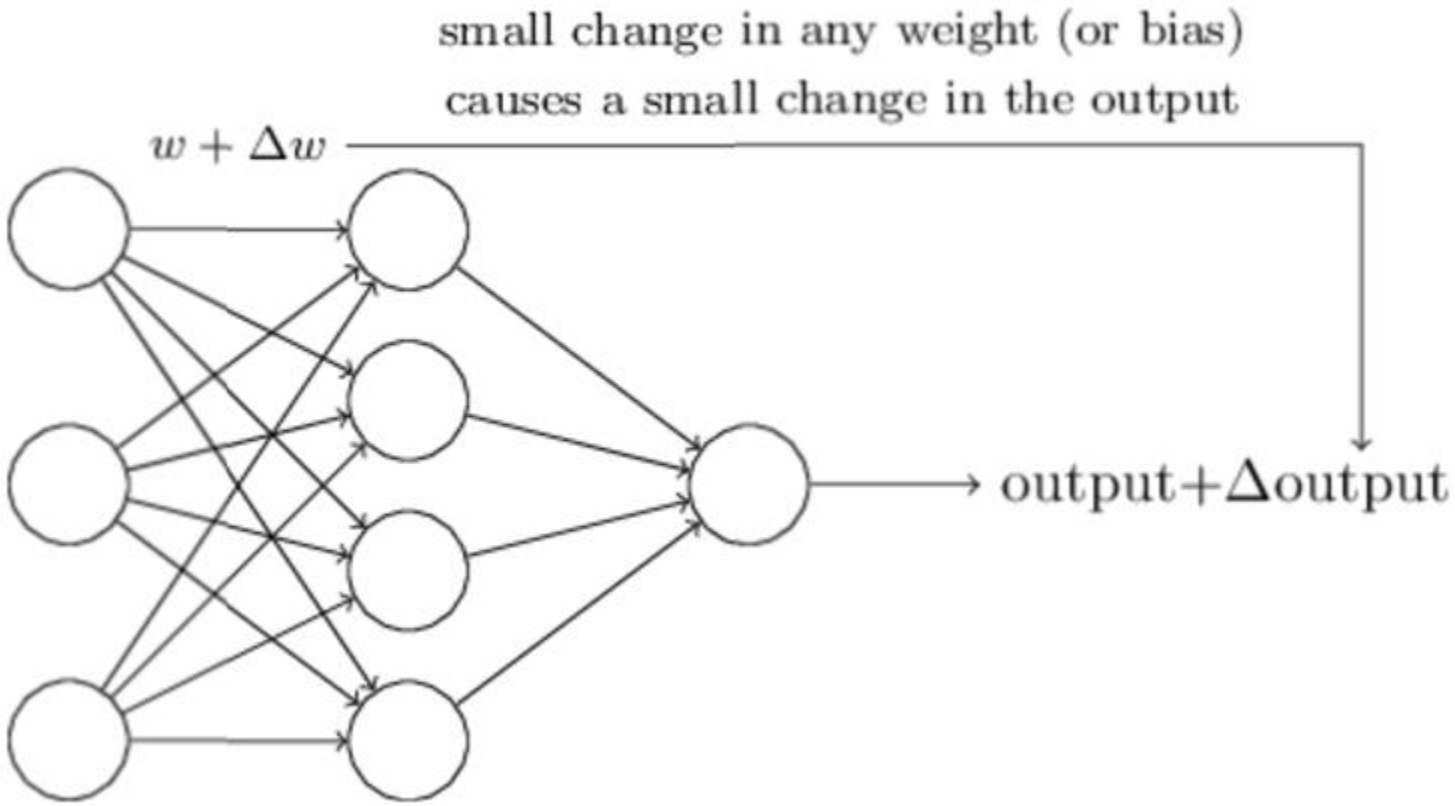
# Why Softened?

- Learning algorithms sound terrific. But how can we devise such algorithms for a neural network? Suppose we have a network of perceptrons that we'd like to use to learn to solve some problem.
- For example, the inputs to the network might be the raw pixel data from a scanned, handwritten image of a digit. And we'd like the network to learn weights and biases so that the output from the network correctly classifies the digit.

# Why?

- To see how learning might work, suppose we make a small change in some weight (or bias) in the network. What we'd like is for this small change in weight to cause only a small corresponding change in the output from the network. As we'll see in a moment, this property will make learning possible. Schematically, here's what we want (obviously this network is too simple to do handwriting recognition!):

# Why?


small change in any weight (or bias)
causes a small change in the output
$w + \Delta w$
output+$\Delta$output

# Why?

- If it were true that a small change in a weight (or bias) causes only a small change in output, then we could use this fact to modify the weights and biases to get our network to behave more in the manner we want. For example, suppose the network was mistakenly classifying an image as an "8" when it should be a "9". We could figure out how to make a small change in the weights and biases so the network gets a little closer to classifying the image as a "9". And then we'd repeat this, changing the weights and biases over and over to produce better and better output. The network would be learning.

# Why?

- The problem is that this isn't what happens when our network contains perceptrons. In fact, a small change in the weights or bias of any single perceptron in the network can sometimes cause the output of that perceptron to completely flip, say from 0 to 1.
- That flip may then cause the behaviour of the rest of the network to completely change in some very complicated way.

# Why?

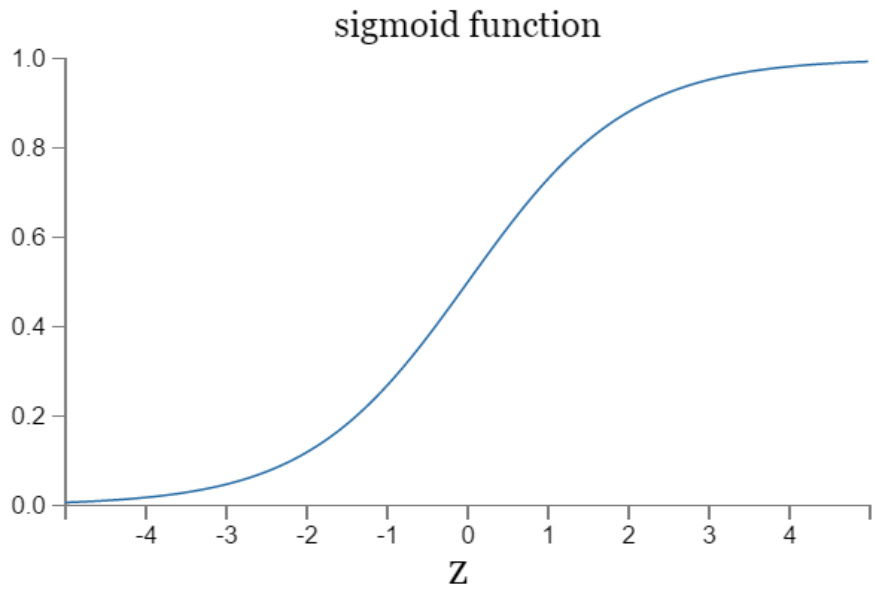
- So while your "9" might now be classified correctly, the behaviour of the network on all the other images is likely to have completely changed in some hard-to-control way. That makes it difficult to see how to gradually modify the weights and biases so that the network gets closer to the desired behavior.

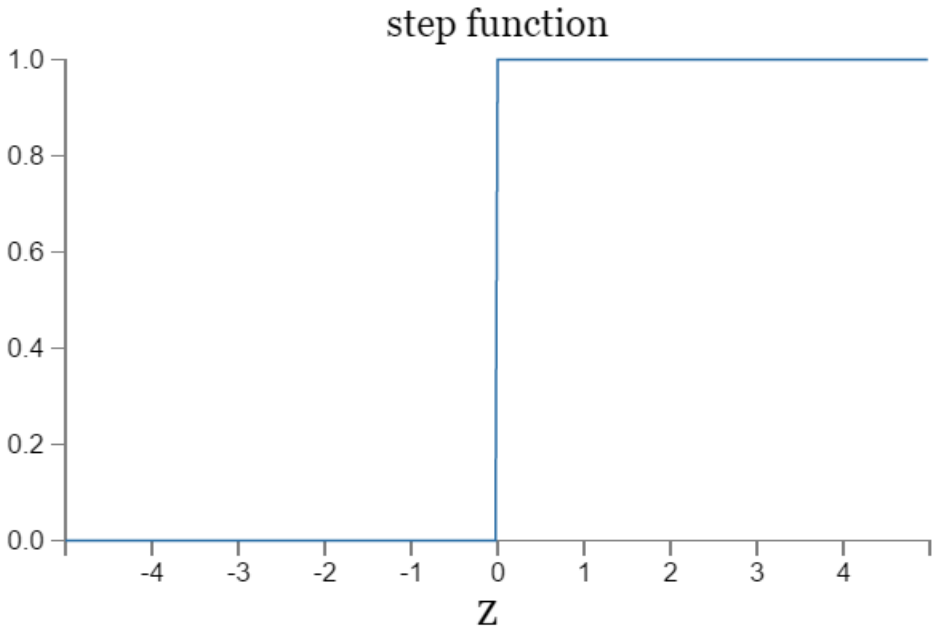
# Sigmoid Neuron

- We can overcome this problem by introducing a new type of artificial neuron called a *sigmoid* neuron. Sigmoid neurons are similar to perceptrons, but modified so that small changes in their weights and bias cause only a small change in their output. That's the crucial fact which will allow a network of sigmoid neurons to learn.
- The output is $\sigma(x) =$,

$$\frac{1}{1+\exp(-\sum_j w_j x_j - b)}.$$

# Sigmoid Neuron

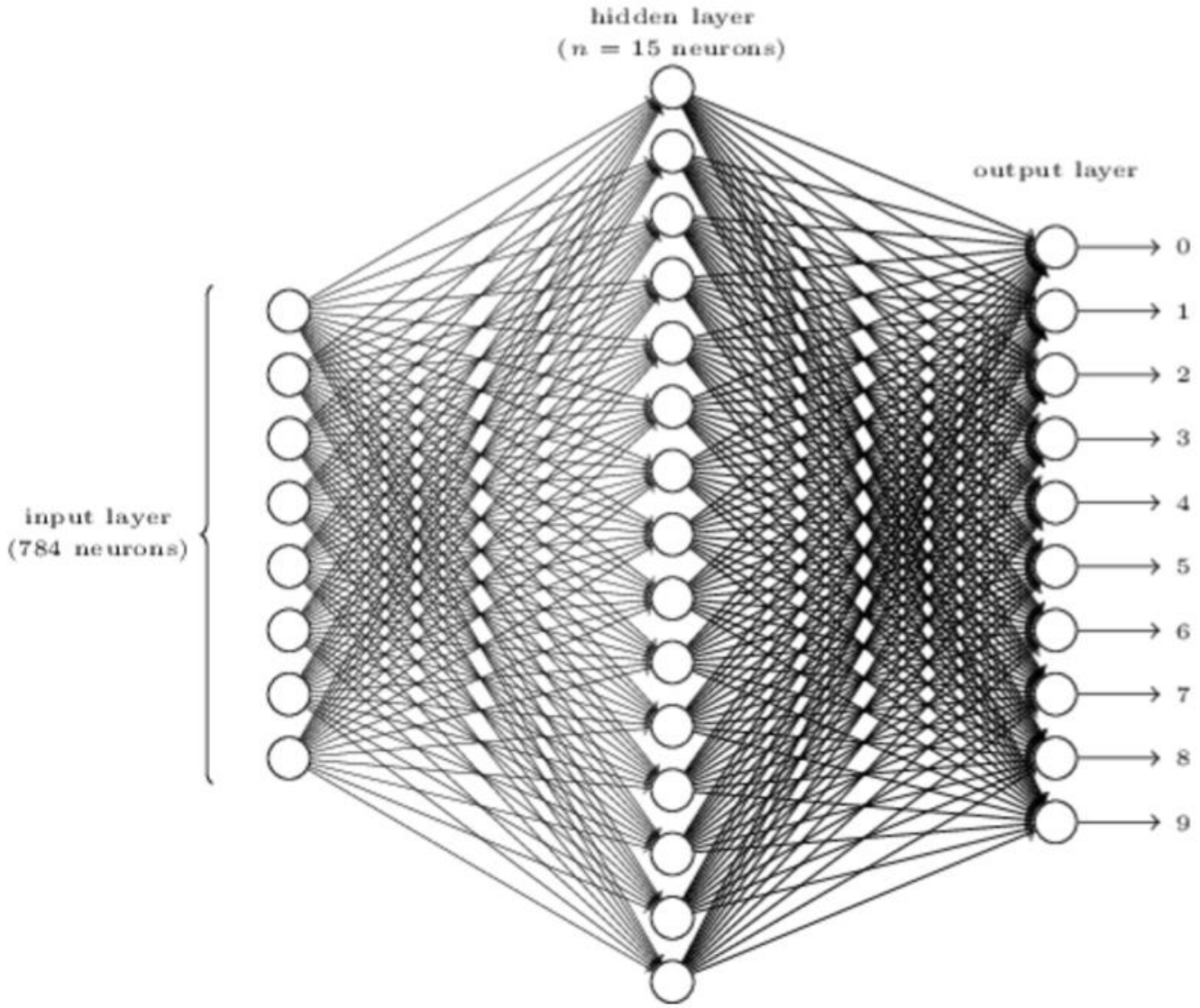


# Forward Propagation

- Given an input X the output of a neural network is computed using `forward propagation'.
- Example:

# An Example of Neural Network



# An Example of Neural Network

- training data for the network will consist of many 28 by 28 pixel images of scanned handwritten digits, and so the input layer contains 784=28×28 neurons.
- The input pixels are greyscale, with a value of 0.0 representing white, a value of 1.0 representing black, and in between values representing gradually darkening shades of grey.
- The second layer of the network is a hidden layer. We denote the number of neurons in this hidden layer by n.

# An Example of Neural Network

- The example shown illustrates a small hidden layer, containing just n=15 neurons.
- The output layer of the network contains 10 neurons. If the first neuron fires, i.e., has an output ≈1, then that will indicate that the network thinks the digit is a 0. If the second neuron fires then that will indicate that the network thinks the digit is a 1. And so on. A little more precisely, we number the output neurons from 0 through 9, and figure out which neuron has the highest activation value. If that neuron is, say, neuron number 6, then our network will guess that the input digit was a 6. And so on for the other output neurons.